



API Mobile Money — Documentation complète

Table des matières

- **0** Authentification
 - 0.1 Login — `POST /api/auth/login`
 - 0.2 Logout — `POST /api/auth/logout`
 - **1** Utilisateur
 - 1.1 Lister tous les utilisateurs — `GET /api/utilisateurs`
 - 1.2 Récupérer un utilisateur — `GET /api/utilisateurs/{id}`
 - 1.3 Créer un utilisateur — `POST /api/utilisateurs`
 - 1.4 Mettre à jour un utilisateur — `PUT /api/utilisateurs/{id}`
 - 1.5 Supprimer un utilisateur — `DELETE /api/utilisateurs/{id}`
 - **2** Compte
 - 2.1 Lister tous les comptes — `GET /api/comptes`
 - 2.2 Récupérer un compte par ID — `GET /api/comptes/{id}`
 - 2.3 Créer un compte — `POST /api/comptes`
 - 2.4 Mettre à jour un compte — `PUT /api/comptes/{id}`
 - 2.5 Supprimer un compte — `DELETE /api/comptes/{id}`
 - 2.6 Lister transactions d'un compte — `GET /api/comptes/{id}/transactions`
 - **3** Transaction
 - Endpoints CRUD et métiers
 - **4** Administration / Stats
 -  Notes générales
-



AUTHENTIFICATION

0.1 Login — `POST /api/auth/login`

Body :

```
{
  "telephone": "771234567",
  "password": "password123"
}
```

Contraintes :

- Vérification du téléphone et mot de passe
- Auth JWT via Laravel Passport
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": 1,
    "nom": "Diallo",
    "prenom": "Moussa",
    "type": "client",
    "links": {
      "self": "/api/utilisateurs/1",
      "comptes": "/api/utilisateurs/1/comptes",
      "transactions": "/api/utilisateurs/1/transactions"
    }
  }
}
```

Réponse erreur (401 Unauthorized) :

```
{
  "status": "error",
  "message": "Identifiants invalides"
}
```

0.2 Logout — POST /api/auth/logout

Contraintes :

- Auth JWT obligatoire
- Invalide le token

Réponse succès (200 OK) :

```
{
  "status": "success",
  "message": "Déconnexion réussie"
}
```

UTILISATEUR

1.1 Lister tous les utilisateurs — GET /api/utilisateurs

Query Params : `page`, `limit`, `sort_by`, `order`, `type`

Contraintes :

- Auth JWT obligatoire
- Admin uniquement
- Pagination et tri obligatoires
- Filtres validés

Réponse succès (200 OK) :

```
{
  "status": "success",
  "page": 1,
  "limit": 10,
  "total": 23,
  "data": [
    {
      "id": 1,
      "nom": "Diallo",
      "prenom": "Moussa",
      "telephone": "771234567",
      "type": "client",
      "links": {
        "self": "/api/utilisateurs/1",
        "comptes": "/api/utilisateurs/1/comptes",
        "transactions": "/api/utilisateurs/1/transactions"
      }
    }
  ]
}
```

Réponse erreur (404) :

```
{
  "status": "error",
  "message": "Aucun utilisateur trouvé pour cette page"
}
```

1.2 Récupérer un utilisateur — GET /api/utilisateurs/{id}**Contraintes :**

- Auth JWT obligatoire
- Admin ou utilisateur propriétaire
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "data": {
    "id": 1,
    "nom": "Diallo",
    "prenom": "Moussa",
    "telephone": "771234567",
    "email": "moussa@example.com",
    "type": "client",
    "comptes": [
      {"id": 101, "numeroCompte": "CPT-001", "solde": 10000}
    ]
  }
}
```

Réponse erreur (404) :

```
{
  "status": "error",
  "message": "Utilisateur non trouvé"
}
```

1.3 Créer un utilisateur — POST /api/utilisateurs

Body: , , , ,

Contraintes :

- Auth JWT obligatoire (admin)
- Vérifier unicité téléphone
- Audit log

Réponse succès (201 Created) :

```
{
  "status": "success",
  "message": "Utilisateur créé",
  "data": {"id": 1, "nom": "Diallo", "prenom": "Moussa", "telephone": "771234567", "type": "client"}
}
```

Réponse erreur (422 / 409) :

```
{
  "status": "error",
```

```
{
  "message": "Téléphone déjà utilisé"
}
```

1.4 Mettre à jour un utilisateur — PUT /api/utilisateurs/{id}

Body : nom, prenom, email

Contraintes :

- Auth JWT obligatoire
- Admin ou propriétaire
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "message": "Utilisateur mis à jour"
}
```

Réponse erreur (422 / 403) :

```
{
  "status": "error",
  "message": "Échec de mise à jour"
}
```

1.5 Supprimer un utilisateur — DELETE /api/utilisateurs/{id}

Contraintes :

- Auth JWT obligatoire (admin ou propriétaire)
- Vérifier que l'utilisateur n'a pas de solde ou compte actif
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "message": "Utilisateur supprimé"
}
```

Réponse erreur (403 / 422) :

```
{
  "status": "error",
  "message": "Suppression impossible"
}
```

COMPTE

2.1 Lister tous les comptes — GET /api/comptes

Query Params : `page`, `limit`, `sort_by`, `order`, `utilisateur_id`, `type`

Contraintes :

- Auth JWT obligatoire
- Admin ou propriétaire
- Pagination, tri et filtres obligatoires
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "page": 1,
  "limit": 10,
  "total": 12,
  "data": [
    {
      "id": 101,
      "numeroCompte": "CPT-001",
      "solde": 10000,
      "type": "courant",
      "utilisateur_id": 1,
      "links": {
        "self": "/api/comptes/101",
        "transactions": "/api/comptes/101/transactions",
        "utilisateur": "/api/utilisateurs/1"
      }
    }
  ]
}
```

Réponse erreur (404) :

```
{
  "status": "error",
```

```
{
  "message": "Aucun compte trouvé pour cette page"
}
```

2.2 Récupérer un compte par ID — GET /api/comptes/{id}

Contraintes :

- Auth JWT obligatoire
- Admin ou propriétaire
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "data": {
    "id": 101,
    "numeroCompte": "CPT-001",
    "solde": 10000,
    "type": "courant",
    "utilisateur": {
      "id": 1,
      "nom": "Diallo",
      "prenom": "Moussa",
      "links": {
        "self": "/api/utilisateurs/1",
        "transactions": "/api/utilisateurs/1/transactions"
      }
    },
    "links": {
      "self": "/api/comptes/101",
      "transactions": "/api/comptes/101/transactions"
    }
  }
}
```

Réponse erreur (404) :

```
{
  "status": "error",
  "message": "Compte non trouvé"
}
```

2.3 Créer un compte — POST /api/comptes

Body: utilisateur_id, type, code_marchand

Contraintes :

- Auth JWT obligatoire (admin ou propriétaire)
- Un client ne peut avoir qu'un compte actif par type
- Commerçants : code_marchand obligatoire et unique
- Audit log

Réponse succès (201 Created) :

```
{
  "status": "success",
  "message": "Compte créé",
  "data": {
    "id": 101,
    "numeroCompte": "CPT-001",
    "solde": 0,
    "type": "courant",
    "utilisateur_id": 1,
    "links": {
      "self": "/api/comptes/101",
      "transactions": "/api/comptes/101/transactions",
      "utilisateur": "/api/utilisateurs/1"
    }
  }
}
```

Réponse erreur (422 / 409) :

```
{
  "status": "error",
  "message": "Impossible de créer le compte, règles métier non respectées"
}
```

2.4 Mettre à jour un compte — PUT /api/comptes/{id}

Body: solde, code_marchand

Contraintes :

- Auth JWT obligatoire (admin ou propriétaire)
- Ne peut modifier solde que via transaction
- Vérifier unicité code_marchand
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "message": "Compte mis à jour",
  "data": {
    "id": 101,
    "solde": 12000,
    "code_marchand": "MCH-001",
    "links": {
      "self": "/api/comptes/101",
      "transactions": "/api/comptes/101/transactions"
    }
  }
}
```

Réponse erreur (422 / 403) :

```
{
  "status": "error",
  "message": "Modification impossible"
}
```

2.5 Supprimer un compte — DELETE /api/comptes/{id}

Contraintes :

- Auth JWT obligatoire (admin ou propriétaire)
- Solde doit être à 0
- Audit log

Réponse succès (200 OK) :

```
{
  "status": "success",
  "message": "Compte supprimé",
  "links": { "utilisateurs": "/api/utilisateurs" }
}
```

Réponse erreur (403 / 422) :

```
{
  "status": "error",
  "message": "Suppression impossible, solde non nul"
}
```

2.6 Lister transactions d'un compte — GET /api/comptes/{id}/transactions

Query Params: `page`, `limit`, `sort_by`, `order`, `type`, `statut`

Contraintes :

- Auth JWT obligatoire
- Admin ou propriétaire du compte
- Pagination et tri obligatoires

Réponse succès (200 OK) :

```
{
  "status": "success",
  "page": 1,
  "limit": 10,
  "total": 5,
  "data": [
    {
      "id": 1001,
      "type": "paiement",
      "montant": 5000,
      "date_transaction": "2025-11-09T13:00:00Z",
      "statut": "success",
      "compte_emetteur": "/api/comptes/101",
      "compte_recepteur": "/api/comptes/102",
      "links": { "self": "/api/transactions/1001" }
    }
  ]
}
```

Réponse erreur (404) :

```
{
  "status": "error",
  "message": "Aucune transaction trouvée"
}
```

TRANSACTION

Toutes les transactions (CRUD + métiers) — dépôts, paiements, transferts — doivent supporter pagination, filtres et HATEOAS.

Endpoints principaux :

- GET /api/transactions
- GET /api/transactions/{id}
- POST /api/transactions
- PUT /api/transactions/{id}/statut
- DELETE /api/transactions/{id}
- POST /api/transactions/depot
- POST /api/transactions/paiement
- POST /api/transactions/transfert

(Voir module Transactions détaillé pour les exemples JSON et contraintes métiers : atomicité, validation, audit logs, idempotence si nécessaire.)



ADMINISTRATION / STATS



Endpoints de statistiques et administration :

- GET /api/stats/solde-total
- GET /api/stats/transactions
- GET /api/utilisateurs/commerçants
- GET /api/utilisateurs/fournisseurs
- GET /api/stats/utilisateur/{id}

(Voir module Stats détaillé pour les exemples JSON et contraintes)



Notes générales

- Tous les endpoints sensibles utilisent JWT Laravel Passport.
- Tous les endpoints CRUD supportent RESTful niveau 4 + HATEOAS (liens dans les réponses).
- Tous les endpoints listant des données utilisent pagination, tri et filtres.
- Toutes les actions critiques sont loggées pour audit.
- Toutes les transactions sont atomiques (débit/crédit).
- Les réponses incluent systématiquement `links` pour navigation HATEOAS.

Annexes / Recommandations d'implémentation

- Utiliser des `FormRequest` Laravel pour centraliser la validation et les messages d'erreur.
- Mettre en place des politiques / gates pour la gestion des autorisations (Admin vs Propriétaire).
- Implémenter un middleware `AuditLog` pour consigner les actions critiques (création/suppression/mise à jour de ressources sensibles).
- Pour les transactions : utiliser des `DB::beginTransaction()` et des verrous si nécessaire pour garantir l'atomicité et prévenir les races conditions.
- Prévoir des tests unitaires et d'intégration pour : authentification, création/suppression de comptes, opérations de transaction (débit/crédit), et endpoints d'administration.

Document généré : version unique README pour l'API Mobile Money — modifiez la section Annexes pour ajouter les exemples de payloads métiers avancés ou spécifier les codes d'erreur internes.